

.B) Ping de test

Créez un fichier testping.py. Afin de pouvoir utiliser SCAPY comme une bibliothèque python ajoutez au début du fichier les lignes suivantes :

```
#!/usr/bin/python
```

```
from scapy.all import *
```

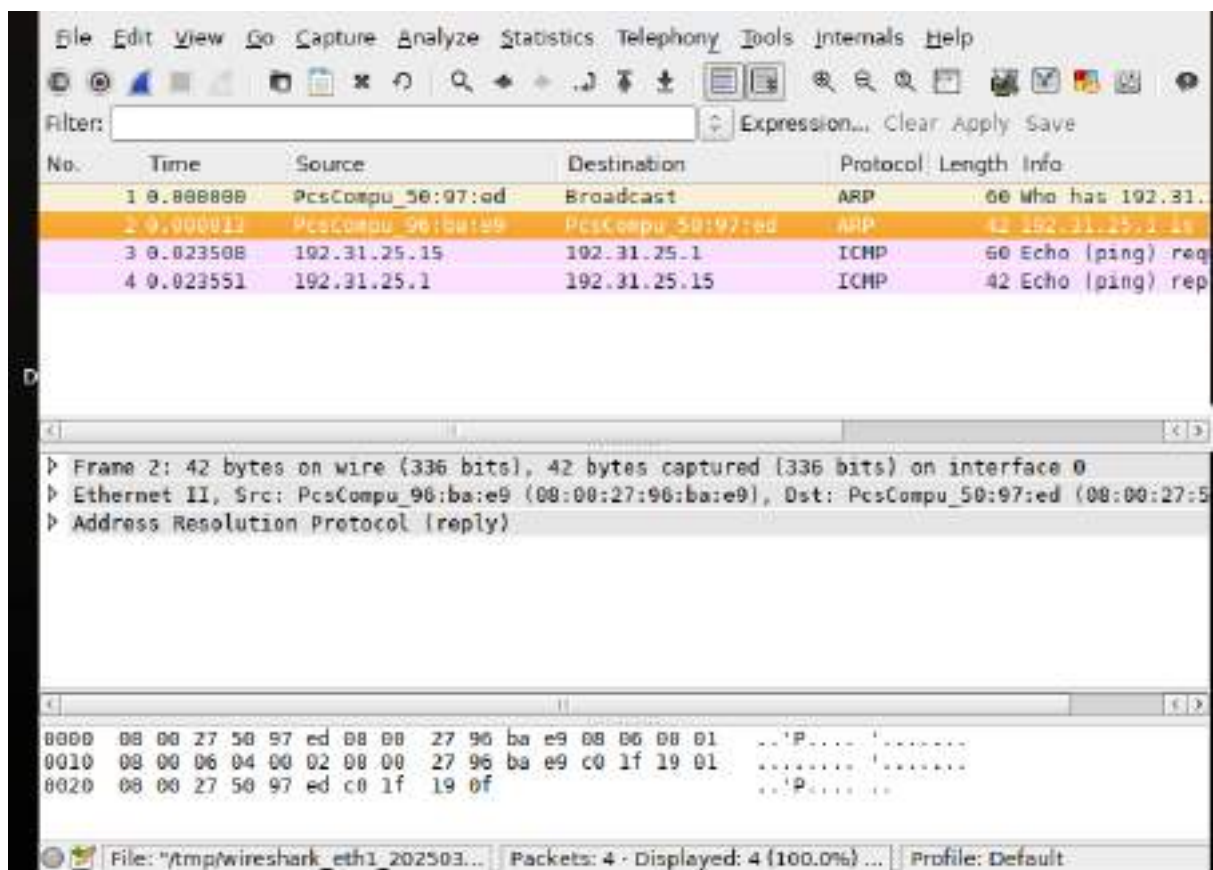
A la suite, ajouter les instructions SCAPY nécessaires pour créer le ping de la question.

Lancez le script testping.py et vérifiez le bon fonctionnement avec Wireshark

```
GNU nano 6.4 testping.py
#!/usr/bin/python
from scapy.all import *

def send_ping(target):
    # Création d'un paquet ICMP Echo Request
    packet = IP(dst=target)/ICMP()
    # Envoi du paquet et attente de la réponse
    response = sr1(packet, timeout=2, verbose=False)
    if response:
        print(f"Réponse reçue de {target}: TTL={response.ttl}")
    else:
        print(f"Aucune réponse de {target}")

if __name__ == "__main__":
    target_ip = "192.31.25.1" # Remplacez par l'adresse IP cible
    send_ping(target_ip)
```



1.C Ping Scan

Créez un ping sweep avec Scapy.

Vérifiez le bon fonctionnement avec Wireshark.

```
#!/usr/bin/python
```

```
from scapy.all import *
```

```
def ping_sweep(network_prefix, start, end):  
    for i in range(start, end + 1):  
        target = f"{network_prefix}.{i}"  
        packet = IP(dst=target)/ICMP()  
        response = sr1(packet, timeout=1, verbose=False)
```

```
    if response:  
        print(f"Hôte actif détecté: {target}")  
    else:  
        print(f"Pas de réponse de {target}")
```

```
if __name__ == "__main__":  
    network_prefix = "192.31.25" # Modifier selon le réseau cible  
    start_ip = 1  
    end_ip = 10 # Modifier selon la plage d'adresses à scanner  
    ping_sweep(network_prefix, start_ip, end_ip)
```

No.	Time	Source	Destination	Protocol	Length	Info
7	0.005190	192.31.25.15	192.31.25.1	ICMP	60	Echo (ping) request id=0x0000, seq=0/0, ttl=64
8	0.005244	192.31.25.1	192.31.25.15	ICMP	42	Echo (ping) reply id=0x0000, seq=0/0, ttl=64
9	0.040830	PcsCompu_50:97:ed	Broadcast	ARP	60	Who has 192.31.25.2? Tell 192.31.25.15
10	0.060930	192.31.25.15	192.31.25.2	ICMP	60	Echo (ping) request id=0x0000, seq=0/0, ttl=64
11	0.120752	PcsCompu_50:97:ed	Broadcast	ARP	60	Who has 192.31.25.3? Tell 192.31.25.15
12	0.135051	192.31.25.15	192.31.25.3	ICMP	60	Echo (ping) request id=0x0000, seq=0/0, ttl=64
13	0.107482	PcsCompu_50:97:ed	Broadcast	ARP	60	Who has 192.31.25.4? Tell 192.31.25.15
14	0.210440	192.31.25.15	192.31.25.4	ICMP	60	Echo (ping) request id=0x0000, seq=0/0, ttl=64
15	0.257701	PcsCompu_50:97:ed	Broadcast	ARP	60	Who has 192.31.25.5? Tell 192.31.25.15
16	0.273111	192.31.25.15	192.31.25.5	ICMP	60	Echo (ping) request id=0x0000, seq=0/0, ttl=64
17	0.320605	PcsCompu_50:97:ed	Broadcast	ARP	60	Who has 192.31.25.6? Tell 192.31.25.15
18	0.347300	192.31.25.15	192.31.25.6	ICMP	60	Echo (ping) request id=0x0000, seq=0/0, ttl=64
19	0.402000	PcsCompu_50:97:ed	Broadcast	ARP	60	Who has 192.31.25.7? Tell 192.31.25.15
20	0.618199	192.31.25.15	192.31.25.7	ICMP	60	Echo (ping) request id=0x0000, seq=0/0, ttl=64
21	1.673715	PcsCompu_50:97:ed	Broadcast	ARP	60	Who has 192.31.25.8? Tell 192.31.25.15
22	1.688545	192.31.25.15	192.31.25.8	ICMP	60	Echo (ping) request id=0x0000, seq=0/0, ttl=64
23	1.756545	PcsCompu_50:97:ed	Broadcast	ARP	60	Who has 192.31.25.9? Tell 192.31.25.15
24	1.66771502	192.31.25.15	192.31.25.9	ICMP	60	Echo (ping) request id=0x0000, seq=0/0, ttl=64
25	1.822244	PcsCompu_50:97:ed	Broadcast	ARP	60	Who has 192.31.25.10? Tell 192.31.25.15
26	1.839399	192.31.25.15	192.31.25.10	ICMP	60	Echo (ping) request id=0x0000, seq=0/0, ttl=64

Frame 32: 324 bytes on wire (2592 bits), 324 bytes captured (2592 bits) on interface 0
Ethernet II, Src: PcsCompu_50:97:ed (08:00:27:50:97:ed), Dst: PcsCompu_06:ba:e0 (08:00:27:06:ba:e0)
Internet Protocol Version 4, Src: 192.31.25.15, Dst: 192.31.25.1

1.D Scan de ports

Créez un script python avec Scapy afin de scanner les ports 22, 80, 53, 161 de l'autre machine.

Vérifiez le bon fonctionnement avec Wireshark

No.	Time	Source	Destination	Protocol	Length	Info
2	0.000019	PcsCompu 98:ba:e8	PcsCompu 50:97:ed	ARP	42	192.31.25.1 is at 08:00:27:9d:ba:e8
3	0.013462	192.31.25.15	192.31.25.1	TCP	60	20 → 22 [SYN] Seq=0 Win=0 Len=0
4	0.013560	192.31.25.1	192.31.25.15	TCP	60	22 → 20 [SYN, ACK] Seq=0 Ack=1 Win=0 Len=0
5	0.015750	192.31.25.15	192.31.25.1	TCP	60	20 → 22 [RST] Seq=1 Win=0 Len=0
6	0.048638	192.31.25.15	192.31.25.1	TCP	60	20 → 22 [RST] Seq=0 Win=0 Len=0
7	0.068236	192.31.25.15	192.31.25.1	TCP	60	20 → 80 [SYN] Seq=0 Win=0 Len=0
8	0.068284	192.31.25.1	192.31.25.15	TCP	60	80 → 20 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
9	0.182875	192.31.25.15	192.31.25.1	TCP	60	20 → 53 [SYN] Seq=0 Win=0 Len=0
10	0.182906	192.31.25.1	192.31.25.15	TCP	60	53 → 20 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
11	0.172448	192.31.25.15	192.31.25.1	TCP	60	20 → 161 [SYN] Seq=0 Win=0 Len=0
12	0.172487	192.31.25.1	192.31.25.15	TCP	60	161 → 20 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0

```
#!/usr/bin/python
```

```
from scapy.all import *
```

```
def port_scan(target, ports):
```

```
    for port in ports:
```

```
        packet = IP(dst=target)/TCP(dport=port, flags="S")
```

```
        response = sr1(packet, timeout=1, verbose=False)
```

```
        if response and response.haslayer(TCP):
```

```
            if response.getlayer(TCP).flags == 0x12: # SYN-ACK reçu
```

```
                print(f"Port {port} ouvert sur {target}")
```

```
                # Envoi d'un RST pour fermer la connexion
```

```
                send(IP(dst=target)/TCP(dport=port, flags="R"), verbose=False)
```

```
            elif response.getlayer(TCP).flags == 0x14: # RST-ACK reçu
```

```
                print(f"Port {port} fermé sur {target}")
```

```
            else:
```

```
                print(f"Pas de réponse du port {port} sur {target}")
```

```
if __name__ == "__main__":
```

```
    target_ip = "192.31.25.1" # Adresse IP de la machine cible
```

```
    ports_to_scan = [22, 80, 53, 161] # Liste des ports à scanner
```

```
    port_scan(target_ip, ports_to_scan)
```

1.E SSH version

Créez un script afin d'obtenir un version SSH de l'autre VM.

Vérifiez le bon fonctionnement avec Wireshark

No.	Time	Source	Destination	Protocol	Length	Info	Information
2	0.000029	192.31.25.1	192.31.25.15	TCP	74	22 → 49592 [SYN, ACK] Seq=0 Ack=1 Win=0 Len=0	
3	0.000456	192.31.25.15	192.31.25.1	TCP	66	49592 → 22 [ACK] Seq=1 Ack=1 Win=64256 Len=0	TcpRst
4	0.001531	192.31.25.1	192.31.25.15	SSH	628	Server: Protocol (SSH-2.0-dropbear_2019.78), Encry	
5	0.001923	192.31.25.15	192.31.25.1	TCP	66	49592 → 22 [ACK] Seq=1 Ack=563 Win=64128 Len=0	TcpRst
6	0.002583	192.31.25.15	192.31.25.1	TCP	66	49592 → 22 [FIN, ACK] Seq=1 Ack=563 Win=64128 Len=0	TcpRst
7	0.002434	192.31.25.1	192.31.25.15	TCP	66	22 → 49592 [FIN, ACK] Seq=301 Ack=2 Win=28992 Len=0	
8	0.002715	192.31.25.15	192.31.25.1	TCP	66	49592 → 22 [ACK] Seq=2 Ack=564 Win=64128 Len=0	TcpRst

```
#!/usr/bin/python
```

```
from scapy.all import *
```

```
def ssh_version_scan(target, port=22):
```

```
    try:
```

```
        # Création d'une connexion TCP
```

```
        sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

```
        sock.settimeout(2)
```

```
        # Tentative de connexion au port SSH
```

```
        sock.connect((target, port))
```

```
        # Réception de la bannière SSH
```

```
        banner = sock.recv(1024).decode().strip()
```

```
        print(f"Bannière SSH de {target}: {banner}")
```

```
        sock.close()
```

```
    except Exception as e:
```

```
        print(f"Impossible d'obtenir la version SSH de {target}: {e}")
```

```
if __name__ == "__main__":
```

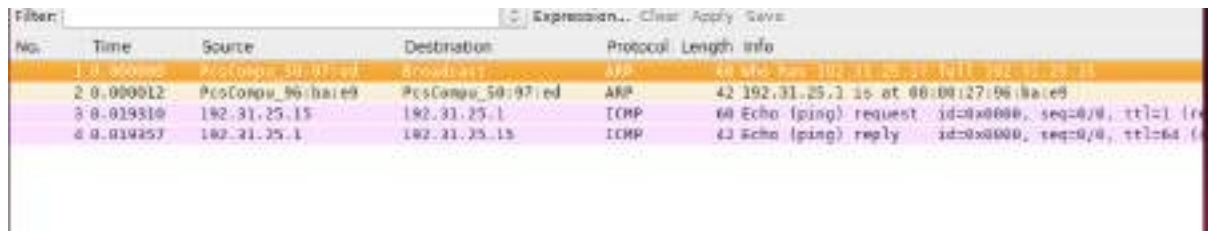
```
    target_ip = "192.31.25.1" # Adresse IP de la machine cible
```

```
    ssh_version_scan(target_ip)
```

1.F Traceroute

Créez un script afin d'obtenir de créer un traceroute avec Scapy.

Vérifiez le bon fonctionnement



No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	ProCompu_50:97:ed	ProCompu_50:97:ed	ARP	42	192.31.25.1 is at 08:00:27:96:b4:e9
2	0.000012	ProCompu_50:b4:e9	ProCompu_50:97:ed	ARP	42	192.31.25.1 is at 08:00:27:96:b4:e9
3	0.019310	192.31.25.15	192.31.25.1	ICMP	60	Echo (ping) request id=0x0000, seq=0/0, ttl=1 (r
4	0.019357	192.31.25.1	192.31.25.15	ICMP	42	Echo (ping) reply id=0x0000, seq=0/0, ttl=64 (r

```
#!/usr/bin/python
```

```
from scapy.all import *
```

```
def traceroute_scan(target, max_hops=30):
```

```
    print(f"Traceroute vers {target}")
```

```
    for ttl in range(1, max_hops + 1):
```

```
        packet = IP(dst=target, ttl=ttl) / ICMP()
```

```
        response = sr1(packet, timeout=2, verbose=False)
```

```
        if response is None:
```

```
            print(f"{ttl}: * Pas de réponse")
```

```
        else:
```

```
            print(f"{ttl}: {response.src}")
```

```
            if response.src == target:
```

```
                print("Arrivée à destination !")
```

```
                break
```

```
if __name__ == "__main__":
```

```
    target_ip = "192.31.25.1" # Adresse IP de la machine cible
```

```
    traceroute_scan(target_ip)
```

```
#!/usr/bin/python
from scapy.all import *

def ip_spoofing(target, spoofed_ip, port=80):
    print(f"Envoi d'un paquet TCP spoofé de {spoofed_ip} vers {target}:{port}")
    packet = IP(src=spoofed_ip, dst=target) / TCP(dport=port, flags="S")
    send(packet, verbose=False)
    print("Paquet envoyé !")

if __name__ == "__main__":
    target_ip = "192.31.25.1" # Adresse IP de la machine cible
    spoofed_ip = "192.168.1.100" # Adresse IP falsifiée
    ip_spoofing(target_ip, spoofed_ip)
```

Filter: Expression... Clear Apply Save						
No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.1.100	192.31.25.1	IP	60	192.31.25.1 > 192.168.1.100: 80 [RST] Seq=192.168.1.100
2	0.000026	PcsCompu_96:b4e9	PcsCompu_59:97ed	ARP	42	192.31.25.1 is at 08:00:27:96:b4e9
3	0.019558	192.168.1.100	192.31.25.1	TCP	60	20 → 80 [SYN] Seq=8192 Len=0

\$